

RTAB-Map Mapping

RTAB-Map Mapping

- 1. Contents
 - 1.1 Introduction to RTAB-Map
 - 1.2. RTAB-Map Working Principle
- 2. Preparation
- 3. Program Startup

1. Contents

This section explains how to combine a vehicle chassis, LiDAR, and a depth camera to implement RTAB-Map mapping.

1.1 Introduction to RTAB-Map

RTAB-MAP (Real-Time Appearance-Based Mapping) is a vision-based real-time simultaneous localization and mapping (SLAM) algorithm primarily used for mapping and navigation tasks in fields such as robotics and augmented reality. RTAB-Map has the following features.

- **Appearance-based loop closure detection:** Primarily uses visual features for location recognition
- **Real-time performance:** Ensures real-time performance through memory management
- **Multi-sensor support:** Supports RGB-D cameras, stereo cameras, and monocular cameras
- **3D map construction:** Creates dense 3D point cloud maps

1.2. RTAB-Map Working Principle

- Front-end processing
 - Acquires image data from sensors
 - Extracts visual features (such as SIFT, SURF, ORB, etc.)
 - Calculates the pose transformation between the current frame and the previous frame
- Back-end optimization
 - Optimizes the pose graph using graph optimization techniques (such as g2o)
 - Detects loop closures (when the robot returns to a previously visited location)
- Memory management
 - Employs a working memory (WM) and long-term memory (LTM) mechanism
 - Recent data is stored in the WM, while older data is transferred to the LTM
 - When a loop closure is detected, the relevant data is transferred from the LTM back to the WM

This section requires entering commands in the terminal. The appropriate terminal will be selected based on your motherboard type. This lesson uses the Raspberry Pi 5 as an example. For Raspberry Pi and Jetson Nano boards, you need to open a terminal on the host computer and enter the command to enter the Docker container. Once inside the Docker container, enter the commands mentioned in this lesson in the terminal. For instructions on entering the Docker container from the host computer, refer to **[01. Robot Configuration and Operation Guide] -- [5.Enter Docker (For JETSON Nano and RPi 5)]**.

For Orin and RDK boards, simply open the terminal and enter the commands mentioned in this lesson.

2. Preparation

Due to performance limitations, the Raspberry Pi 5 and Jetson Nano cannot smoothly run the RTAB-Map algorithm in Docker on the board. Therefore, a virtual machine is required to facilitate this. To enable distributed communication between the car and the virtual machine, two steps are required:

- Both must be on the same local area network. This is most easily achieved by connecting to the same Wi-Fi network.
- Both devices must have the same ROS_DOMAIN_ID. The default ROS_DOMAIN_ID for the car is 30, and the default ROS_DOMAIN_ID for the virtual machine is also 30. If they are different, modify the virtual machine's ROS_DOMAIN_ID. To do this, modify the ~/.bashrc file and change the ROS_DOMAIN_ID value to match the car's. Save and exit the file, then enter the command source ~/.bashrc to refresh the environment variables.
- To verify distributed communication between the two, run the driver program on the board, then enter ros2 node list on the virtual machine. If the underlying data topic appears, the two devices are communicating.

The Orin board can be run directly on the board.

3. Program Startup

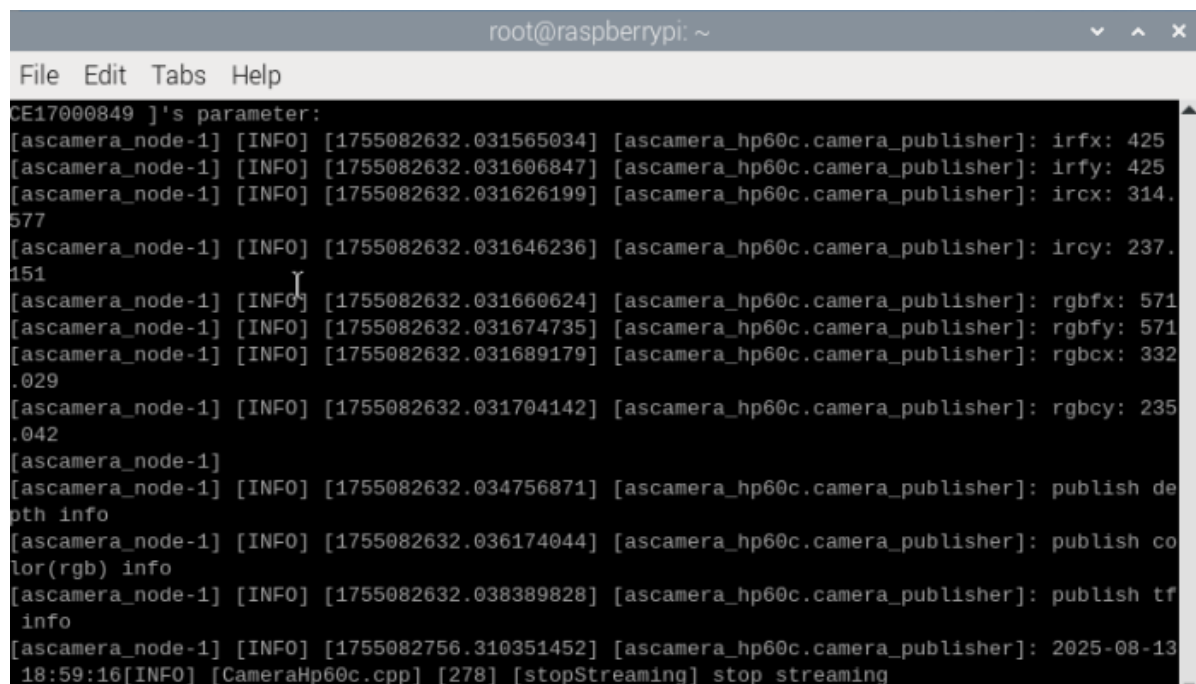
For PI5/JETSON NANO controllers, you need to enter a Docker container first; for RDK X5 and Orin boards, this is not required.

Entering a Docker container (see the Docker course for steps: 4. Docker Startup Script).

All commands below must be executed within the same Docker container (see the Docker course for steps: 3. Docker Commit and Multi-Terminal Entry).

First, enter the following command in the car terminal to start the depth camera.

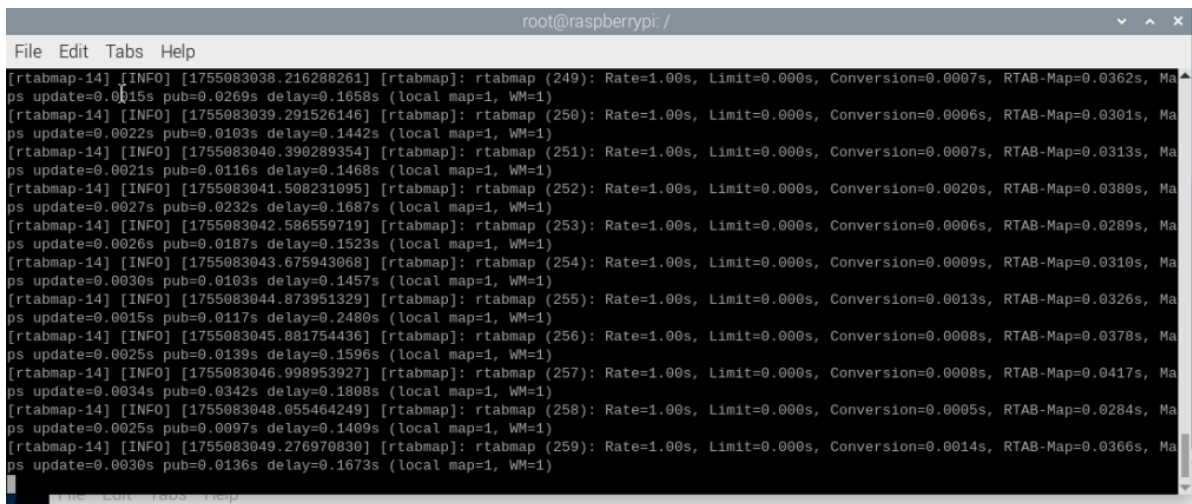
```
ros2 launch ascamera hp60c.launch.py
```



```
root@raspberrypi: ~
File Edit Tabs Help
[ascamera_node-1] [INFO] [1755082632.031565034] [ascamera_hp60c.camera_publisher]: irfx: 425
[ascamera_node-1] [INFO] [1755082632.031606847] [ascamera_hp60c.camera_publisher]: irfy: 425
[ascamera_node-1] [INFO] [1755082632.031626199] [ascamera_hp60c.camera_publisher]: ircx: 314.
577
[ascamera_node-1] [INFO] [1755082632.031646236] [ascamera_hp60c.camera_publisher]: ircy: 237.
151
[ascamera_node-1] [INFO] [1755082632.031660624] [ascamera_hp60c.camera_publisher]: rgbfx: 571
[ascamera_node-1] [INFO] [1755082632.031674735] [ascamera_hp60c.camera_publisher]: rgbfy: 571
[ascamera_node-1] [INFO] [1755082632.031689179] [ascamera_hp60c.camera_publisher]: rgbcx: 332
.029
[ascamera_node-1] [INFO] [1755082632.031704142] [ascamera_hp60c.camera_publisher]: rgbcy: 235
.042
[ascamera_node-1]
[ascamera_node-1] [INFO] [1755082632.034756871] [ascamera_hp60c.camera_publisher]: publish de
pth info
[ascamera_node-1] [INFO] [1755082632.036174044] [ascamera_hp60c.camera_publisher]: publish co
lor(rgb) info
[ascamera_node-1] [INFO] [1755082632.038389828] [ascamera_hp60c.camera_publisher]: publish tf
info
[ascamera_node-1] [INFO] [1755082756.310351452] [ascamera_hp60c.camera_publisher]: 2025-08-13
18:59:16[INFO] [CameraHp60c.cpp] [278] [stopStreaming] stop streaming
```

Enter the following command in the car terminal to start mapping, chassis, and radar.

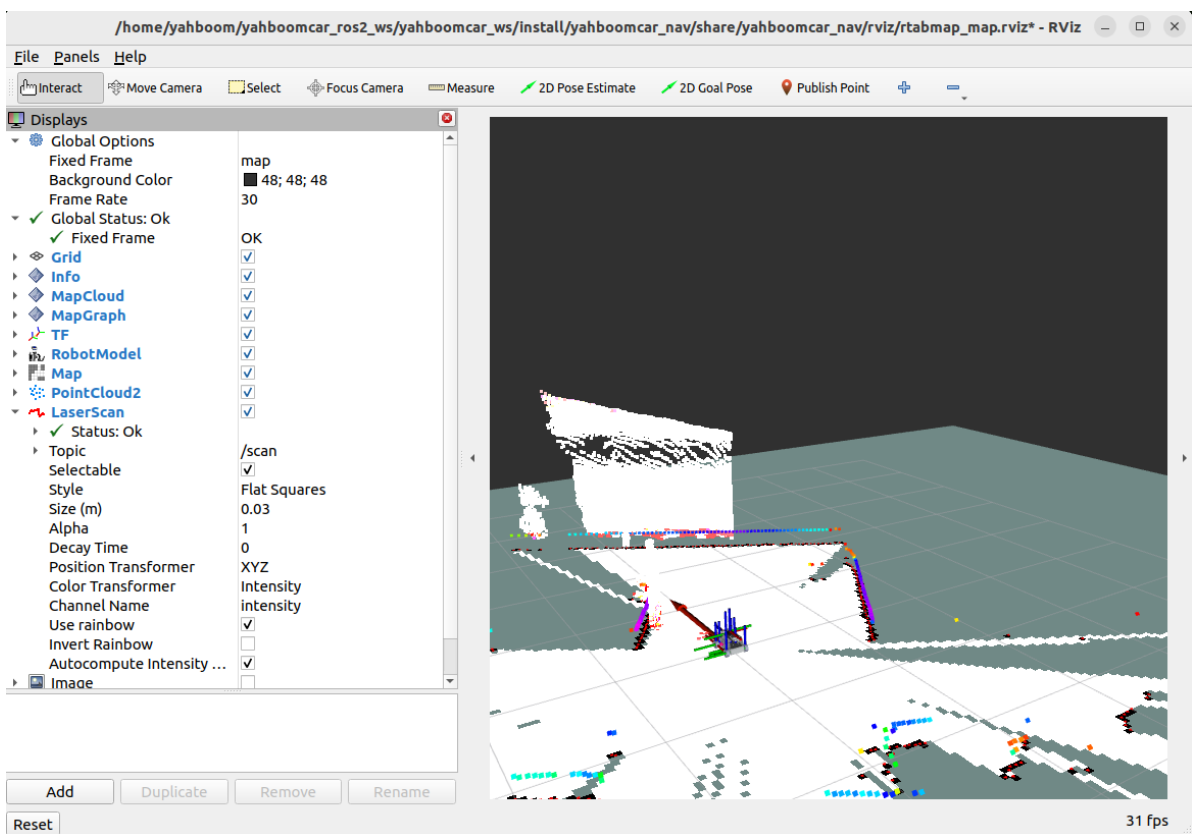
```
ros2 launch yahboomcar_nav map_rtabmap_launch.py
```



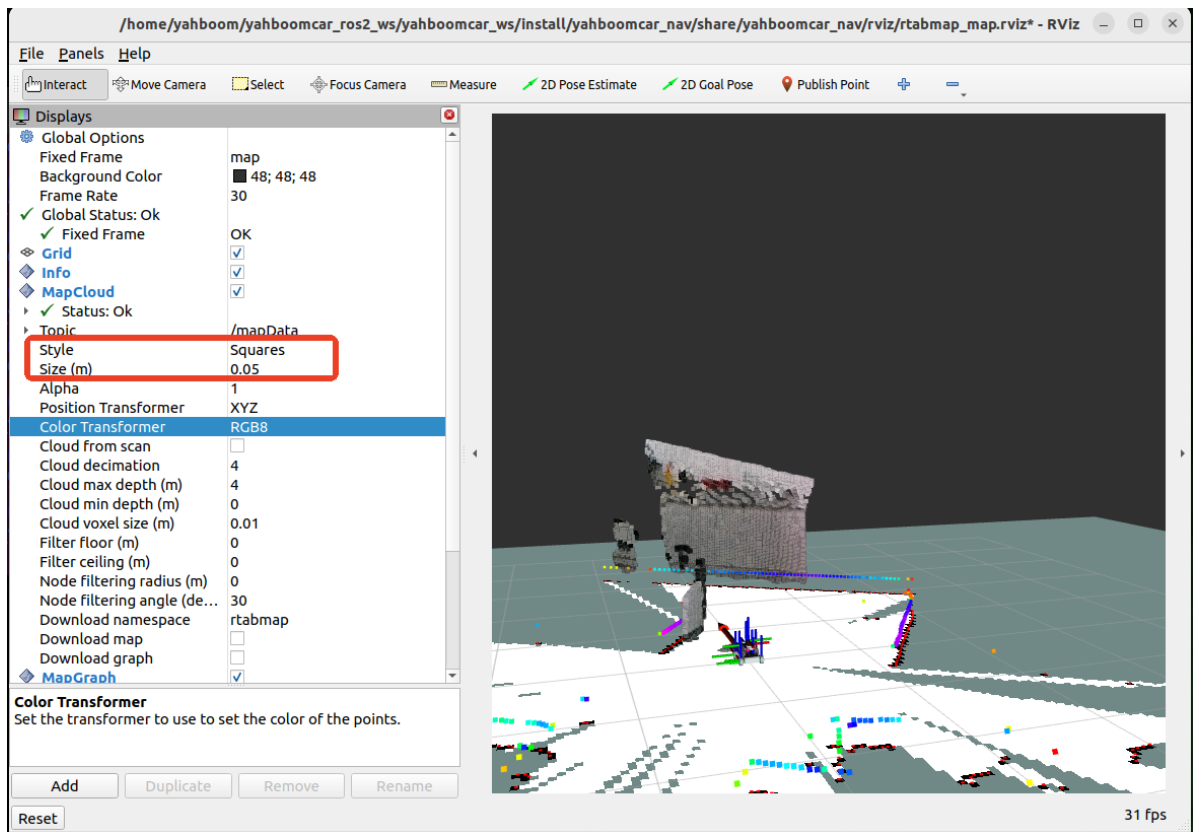
Enter the following command on the virtual machine to perform rviz visualization.

```
ros2 launch yahboomcar_nav display_rtabmap_map_launch.py
```

After successful startup, the image below will appear.

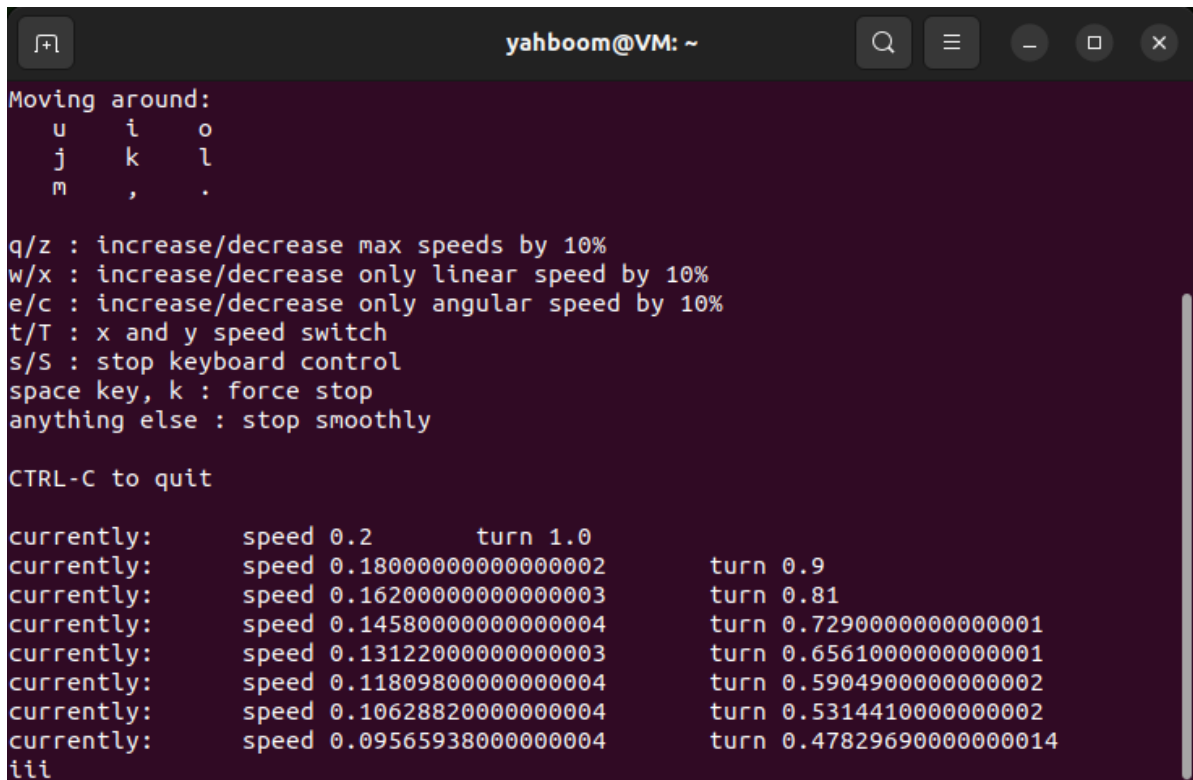


Modify the settings in rviz to change the point cloud display to RGB, as shown below.



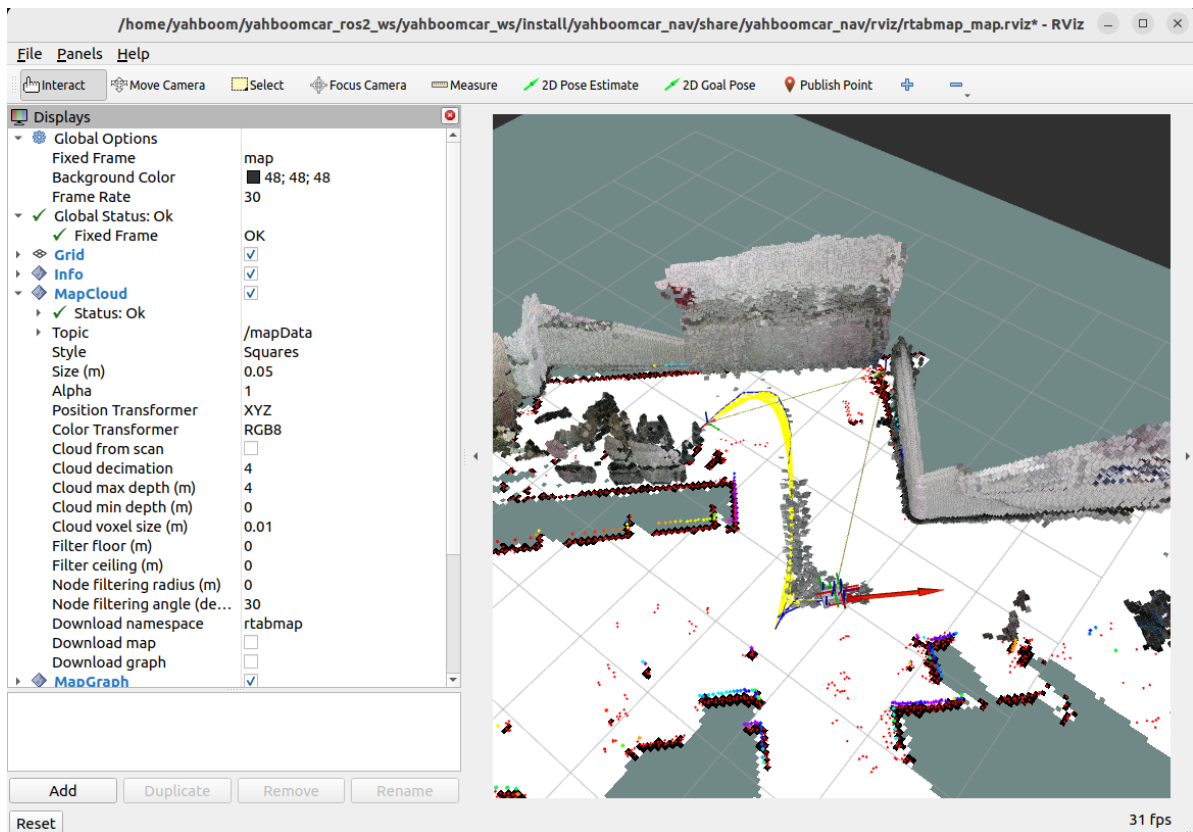
Finally, open a terminal in the virtual machine and enter the following command to control the car's movement and map creation using the keyboard.

```
ros2 run yahboomcar_ctrl yahboom_keyboard
```



Here, control the car's movement as slowly as possible. Press the [z] key to reduce the linear velocity and deceleration. Then, press the [i] key to move the car forward, the [,] key to move it backward, the [j] key to rotate the car left, and the [l] key to rotate the car right.

As shown below, the map is created.



After the map is created, press Ctrl+C in the terminal where you launched map_rtabmap_launch.py to close the program. The map will be saved in /root/.ros/rtabmap.db.

```

File Edit Tabs Help
root@raspberrypi: /

[ydlidar_ros2_driver_node-9] [YDLIDAR] Thread [0x82E5E8E0] has been canceled
[rtabmap-14] [INFO] [1755083876.801850326] [rtabmap]: Parameters are not saved (No configuration file provided...)
[ydlidar_ros2_driver_node-9] [YDLIDAR] Now lidar scanning has stopped!
[INFO] [ydlidar_ros2_driver_node-9]: process has finished cleanly [pid 294819]
[rtabmap-14] [WARN] (2025-08-13 19:13:28.828) MapsManager.cpp:863::publishMaps() Graph has changed! The whole cloud is regenerated.
[rtabmap-14] [WARN] (2025-08-13 19:13:33.297) Rtabmap.cpp:4591::process() Republishing data of requested node(s) 1 (Rtabmap/MaxRepubli
shed=2)
[rtabmap-14] [WARN] (2025-08-13 19:13:50.859) MapsManager.cpp:863::publishMaps() Graph has changed! The whole cloud is regenerated.
[rtabmap-14] [WARN] (2025-08-13 19:13:51.939) Rtabmap.cpp:4591::process() Republishing data of requested node(s) 1 (Rtabmap/MaxRepubli
shed=2)
[rtabmap-14] [WARN] (2025-08-13 19:15:08.804) Rtabmap.cpp:3067::process() Rejected loop closure 40 -> 94: Not enough inliers 0/40 (mat
ches=18) between 40 and 94
[rtabmap-14] [WARN] (2025-08-13 19:15:11.018) Rtabmap.cpp:3067::process() Rejected loop closure 40 -> 96: Not enough inliers 0/40 (mat
ches=11) between 40 and 96
[rtabmap-14] [WARN] (2025-08-13 19:15:16.690) Rtabmap.cpp:3067::process() Rejected loop closure 40 -> 101: Not enough inliers 0/40 (ma
tches=37) between 40 and 101
[rtabmap-14] [WARN] (2025-08-13 19:15:31.856) Rtabmap.cpp:3067::process() Rejected loop closure 51 -> 115: Not enough inliers 0/40 (ma
tches=16) between 51 and 115
[rtabmap-14] rtabmap: Saving database/long-term memory... (located at /root/.ros/rtabmap.db)
[rtabmap-14] rtabmap: Saving database/long-term memory...done! (located at /root/.ros/rtabmap.db, 51 MB)
[INFO] [rtabmap-14]: process has finished cleanly [pid 294834]
root@raspberrypi: /#

```

```

root@raspberrypi:~# cd .ros/
root@raspberrypi:~/.ros# ls
log rosdep rtabmap.db
root@raspberrypi:~/.ros#

```

An orange arrow points to the file 'rtabmap.db' in the directory listing.